

Universidade Federal de Alfenas

Instituto de Ciências Exatas

Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Sistemas de Equações Lineares:

Métodos Diretos em Python

Disciplina: Cálculo Numérico

Professora: Angela Leite Moreno

Modalidade: Atividade de investigação em dupla

Entrega: Relatório .pdf + código-fonte .py

Apresentação

Este plano de investigação propõe uma série de experimentos computacionais que esperam levar o aluno a **compreender de dentro para fora** os principais métodos diretos para solução de sistemas lineares $Ax = b$. O código-fonte de cada método já está disponível; cabe ao aluno *executar, modificar, medir, analisar e interpretar*.

Objetivos

Ao concluir esta investigação, o aluno será capaz de:

- a) Implementar e aplicar a Eliminação de Gauss com pivoteamento parcial;
- b) Construir e utilizar a Fatoração LU (Doolittle) e a Fatoração de Cholesky;
- c) Aplicar o Algoritmo de Thomas a sistemas tridiagonais;
- d) Medir e comparar empiricamente o custo computacional ($O(n^3)$ vs. $O(n)$);
- e) Analisar o impacto do condicionamento e do pivoteamento na precisão;
- f) Tomar decisões fundamentadas sobre qual método usar em cada situação.

Atenção

O objetivo **não** é apenas obter resultados corretos, mas entender *por que* cada método funciona (ou falha). Documente todas as observações, mesmo quando inesperadas.

Sumário

Apresentação	1
1 Eliminação de Gauss e Pivoteamento	3
1.1 Verificação básica	4
1.2 Efeito do pivoteamento na precisão	4
1.3 Sistemas singulares e quase-singulares	4
2 Fatoração LU (Doolittle)	5
2.1 Construção e verificação	6
2.2 Vantagem com múltiplos lados direitos	6
2.3 Fatoração PLU (com pivoteamento)	6
3 Fatoração de Cholesky para Matrizes SPD	7
3.1 Identificando matrizes SPD	8
3.2 Cholesky vs. LU: custo	8
4 Algoritmo de Thomas para Sistemas Tridiagonais	9
4.1 Execução e verificação	10
4.2 Thomas vs. Gauss: escalonamento	10
5 Custo Computacional Empírico	11
5.1 Lei de escala	11
5.2 Comparação de métodos	11
6 Condicionamento e Sensibilidade à Perturbação	12
6.1 Matriz de Hilbert	12
6.2 Amplificação de erros	13
7 Projeto Integrador: PageRank Numérico	14
8 Desafio (Opcional — Pontuação Extra)	14
Entrega e Critérios de Avaliação	15
A Esqueleto do Script Principal	16
B Referências Bibliográficas	17

1. Eliminação de Gauss e Pivoteamento

Dica

O código a seguir implementa a Eliminação de Gauss *com* pivoteamento parcial e a substituição retroativa. Copie-o para o arquivo `gauss.py` e **não o modifique** antes de responder as questões desta seção.

```
1 import numpy as np
2
3 def gauss(A, b):
4     """Elimina o de Gauss com pivoteamento parcial.
5     Retorna (A_triangular, b_modificado).
6     """
7     A = np.array(A, dtype=float)
8     b = np.array(b, dtype=float)
9     n = len(b)
10
11     for k in range(n - 1):
12         # Pivoteamento parcial: busca maior |a_ik| abaixo da
13         # linha k
14         p = np.argmax(np.abs(A[k:, k])) + k
15         A[[k, p]] = A[[p, k]]
16         b[[k, p]] = b[[p, k]]
17
18         for i in range(k + 1, n):
19             m = A[i, k] / A[k, k]           # multiplicador
20             A[i, k:] -= m * A[k, k:]
21             b[i] -= m * b[k]
22
23     return A, b
24
25 def subst_retro(A, b):
26     """Substitui o retroativa para sistema triangular
27     superior."""
28     n = len(b)
29     x = np.zeros(n)
30     for i in range(n - 1, -1, -1):
31         x[i] = (b[i] - A[i, i+1:] @ x[i+1:]) / A[i, i]
32     return x
33
34 def resolver_gauss(A, b):
35     """Pipeline completo: Gauss + substitui o retroativa."""
36     Au, bu = gauss(A, b)
37     return subst_retro(Au, bu)
```

Listing 1: `gauss.py` — Eliminação de Gauss com pivoteamento parcial

1.1. Verificação básica

Q1.1. Execute o código acima para o sistema da aula:

$$3x_1 + 2x_2 + 4x_3 = 1, \quad x_1 + x_2 + 2x_3 = 2, \quad 4x_1 + 3x_2 - 2x_3 = 3.$$

Verifique que a solução obtida é $x = (-3, 5, 0)^T$. Calcule o **resíduo** $r = \|Ax - b\|_2$ e interprete o resultado.

Q1.2. Imprima a matriz triangular superior U gerada após a eliminação. Compare cada elemento com os valores apresentados na aula. Qual linha foi trocada pelo pivoteamento?

1.2. Efeito do pivoteamento na precisão

Q1.3. Implemente uma versão **sem** pivoteamento (remova as linhas de troca da função **gauss**). Aplique ambas as versões ao sistema mal-condicionado

$$\begin{pmatrix} 0,0003 & 3 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2,0001 \\ 1 \end{pmatrix},$$

cujas soluções exatas são $(1/3, 2/3)^T$. Use aritmética **float32** (`dtype=np.float32`) para simular precisão reduzida. Preencha a tabela:

Método	x_1 calculado	Erro relativo x_1	Erro relativo x_2
Sem pivoteamento			
Com piv. parcial			

Explique por que o pivoteamento reduz drasticamente o erro.

Q1.4. Repita o experimento anterior variando o coeficiente a_{11} entre $10^{-1}, 10^{-3}, 10^{-6}, 10^{-9}$. Plote o erro relativo em x_1 em função de a_{11} (escala log-log) para as duas versões. O que você observa?

1.3. Sistemas singulares e quase-singulares

Q1.5. Aplique **resolver_gauss** ao sistema singular (do exercício da aula):

$$x - 3y + z = 1, \quad 6x - 18y + 4z = 2, \quad -x + 3y - z = 4.$$

O que acontece? Modifique o código para **detectar** e reportar singularidade (pivô nulo ou muito pequeno). Use como limiar $|a_{kk}| < 10^{-12}$.

2. Fatoração LU (Doolittle)

```
1 import numpy as np
2
3 def fatoracao_lu(A):
4     """Fatoração de Doolittle: A = LU sem pivoteamento.
5     Retorna (L, U).
6     """
7     A = np.array(A, dtype=float)
8     n = A.shape[0]
9     L = np.eye(n)
10    U = np.zeros((n, n))
11
12    for k in range(n):
13        # Linha k de U
14        for j in range(k, n):
15            U[k, j] = A[k, j] - L[k, :k] @ U[:k, j]
16        # Coluna k de L
17        for i in range(k + 1, n):
18            L[i, k] = (A[i, k] - L[i, :k] @ U[:k, k]) / U[k, k]
19
20    return L, U
21
22
23 def subst_prog(L, b):
24     """Substitui o progressiva: resolve Ly = b."""
25     n = len(b)
26     y = np.zeros(n)
27     for i in range(n):
28         y[i] = b[i] - L[i, :i] @ y[:i]
29     return y
30
31
32 def subst_retro(U, y):
33     """Substitui o retroativa: resolve Ux = y."""
34     n = len(y)
35     x = np.zeros(n)
36     for i in range(n - 1, -1, -1):
37         x[i] = (y[i] - U[i, i+1:] @ x[i+1:]) / U[i, i]
38     return x
39
40
41 def resolver_lu(A, b):
42     """Resolve Ax = b via fatoração LU."""
43     L, U = fatoracao_lu(A)
44     y = subst_prog(L, b)
45     x = subst_retro(U, y)
46     return x, L, U
```

Listing 2: lu.py — Fatoração LU de Doolittle e resolução

2.1. Construção e verificação

Q2.1. Para a matriz

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix},$$

obtenha L e U usando `fatoracao_lu`. Calcule o erro de fatoração $\|LU - A\|_F$ (norma de Frobenius). Imprima L e U formatados e confirme visualmente a estrutura triangular.

Q2.2. Resolva o mesmo sistema para dois lados direitos distintos: $b_1 = (1, 2, 3)^T$ e $b_2 = (0, 1, -1)^T$. **Fatore A apenas uma vez** e reutilize L e U para ambos. Verifique os resíduos $\|Ax_i - b_i\|_2$.

2.2. Vantagem com múltiplos lados direitos

Q2.3. Crie uma matriz aleatória $A \in \mathbb{R}^{n \times n}$ (com $n = 200$, semente `np.random.seed(42)`) e $k = 50$ vetores b aleatórios. Meça o tempo de execução de duas estratégias:

- (a) **Gauss repetido:** chamar `resolver_gauss(A, b)` para cada b ;
- (b) **LU reutilizado:** fatorar A uma vez e resolver os k sistemas via substituições progressiva/retroativa.

Use `time.perf_counter()` para medir. Preencha a tabela:

Estratégia	Tempo total (s)	Tempo por sistema (ms)
Gauss repetido		
LU reutilizado		

Qual é o fator de aceleração? Justifique teoricamente com base na contagem de flops.

2.3. Fatoração PLU (com pivoteamento)

Q2.4. A função `scipy.linalg.lu` retorna P, L, U tal que $PA = LU$. Para a matriz

$$A = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix},$$

compare: (a) `fatoracao_lu` (falha?); (b) `scipy.linalg.lu`. Explique o papel da matriz de permutação P . Como se usa P para resolver $Ax = b$?

3. Fatoração de Cholesky para Matrizes SPD

```

1 import numpy as np
2
3 def cholesky(A):
4     """Fatoração de Cholesky:  $A = L @ L.T$ 
5     Requer A simétrica positiva definida (SPD).
6     Lança ValueError se A não for SPD.
7     """
8     A = np.array(A, dtype=float)
9     n = A.shape[0]
10    L = np.zeros((n, n))
11
12    for k in range(n):
13        soma_diag = A[k, k] - np.sum(L[k, :k] ** 2)
14        if soma_diag <= 0:
15            raise ValueError(
16                f"Matriz não SPD: elemento diagonal {k}
17                negativo ({soma_diag:.4e})"
18            )
19        L[k, k] = np.sqrt(soma_diag)
20        for i in range(k + 1, n):
21            L[i, k] = (A[i, k] - np.sum(L[i, :k] * L[k, :k])) /
22                L[k, k]
23
24    return L
25
26 def resolver_cholesky(A, b):
27     """Resolve  $Ax = b$  via Cholesky (A deve ser SPD)."""
28     L = cholesky(A)
29     #  $Ly = b$  (substituição progressiva)
30     n = len(b)
31     y = np.zeros(n)
32     for i in range(n):
33         y[i] = (b[i] - L[i, :i] @ y[:i]) / L[i, i]
34     #  $L^T x = y$  (substituição retroativa)
35     x = np.zeros(n)
36     Lt = L.T
37     for i in range(n - 1, -1, -1):
38         x[i] = (y[i] - Lt[i, i+1:] @ x[i+1:]) / Lt[i, i]
39     return x, L

```

Listing 3: cholesky.py — Fatoração de Cholesky

3.1. Identificando matrizes SPD

Q3.1. Teste `cholesky` nas três matrizes abaixo. Antes de executar, **preveja** qual será o resultado de cada uma:

$$A_1 = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}, \quad A_2 = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}, \quad A_3 = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 3 & 0 \\ 2 & 0 & 3 \end{pmatrix}.$$

Para cada matriz: (a) verifique a previsão; (b) calcule os autovalores com `np.linalg.eigvalsh` e relacione com o critério SPD.

3.2. Cholesky vs. LU: custo

Q3.2. Para matrizes SPD aleatórias de tamanho $n \in \{50, 100, 200, 500\}$ (geradas como $A = B^T B + nI$ para B aleatória), meça o tempo de Cholesky versus LU. Plote tempo $\times n$ em escala log-log. Confirme empiricamente a relação de $\approx 2\times$ a favor de Cholesky.

Q3.3. Na regressão linear, as **equações normais** têm a forma $(X^T X)\beta = X^T y$, onde $X^T X$ é sempre SPD quando X tem colunas linearmente independentes. Gere uma matriz de design $X \in \mathbb{R}^{100 \times 5}$ (com coluna de uns e 4 colunas aleatórias), $y = X\beta^* + \varepsilon$ com $\beta^* = (2, -1, 3, 0.5, -2)^T$ e ruído gaussiano. Resolva as equações normais via Cholesky e compare $\hat{\beta}$ com `np.linalg.lstsq`.

4. Algoritmo de Thomas para Sistemas Tridiagonais

```

1 import numpy as np
2
3 def thomas(a, b, c, d):
4     """Resolve sistema tridiagonal pelo Algoritmo de Thomas.
5
6     Par metros
7     -----
8     a : subdiagonal (comprimento n-1; a[0] = elemento (2,1))
9     b : diagonal principal (comprimento n)
10    c : superdiagonal (comprimento n-1; c[0] = elemento (1,2))
11    d : lado direito (comprimento n)
12
13    Retorna
14    -----
15    x : vetor solu o (comprimento n)
16    """
17    n = len(b)
18    # C opias para n o modificar os vetores originais
19    b = np.array(b, dtype=float)
20    c = np.array(c, dtype=float)
21    d = np.array(d, dtype=float)
22
23    # Etapa de elimina o progressiva
24    for k in range(1, n):
25        m = a[k - 1] / b[k - 1] # multiplicador
26        b[k] -= m * c[k - 1]
27        d[k] -= m * d[k - 1]
28
29    # Substitui o retroativa
30    x = np.zeros(n)
31    x[-1] = d[-1] / b[-1]
32    for k in range(n - 2, -1, -1):
33        x[k] = (d[k] - c[k] * x[k + 1]) / b[k]
34
35    return x
36
37
38 def montar_tridiagonal(a, b, c):
39     """Constr i matriz densa a partir das diagonais (para
40     verifica o)."""
41     n = len(b)
42     A = np.diag(b) + np.diag(a, -1) + np.diag(c, 1)
43     return A

```

Listing 4: thomas.py — Algoritmo de Thomas $O(n)$

4.1. Execução e verificação

Q4.1. Resolva o sistema tridiagonal 5×5 :

$$\begin{pmatrix} 4 & -1 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & -1 & 4 \end{pmatrix} x = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

Verifique o resíduo $\|Ax - d\|_2$ construindo A com `montar_tridiagonal`. Este sistema surge na discretização de que tipo de equação diferencial?

4.2. Thomas vs. Gauss: escalonamento

Q4.2. Para $n \in \{100, 500, 1000, 5000, 10000\}$, gere sistemas tridiagonais com $b_i = 4$, $a_i = c_i = -1$ e $d_i = 1$. Meça o tempo de solução usando: (a) `thomas`; (b) `resolver_gauss` (matriz densa). Preencha a tabela e plote tempo $\times n$:

n	Thomas (ms)	Gauss (ms)	Razão	Razão teórica
100				
500				
1000				
5000				
10000				

Qual a relação entre os tempos observados? Ela cresce como previsto pela teoria ($O(n^3)$ vs. $O(n)$)?

Q4.3. Para $n = 10000$, qual é o **tamanho em megabytes** que a matriz densa ocuparia em memória (`float64`)? E a representação tridiagonal com três vetores? Use $\text{MB} = n^2 \times 8/2^{20}$. Discuta as implicações para aplicações reais.

5. Custo Computacional Empírico

5.1. Lei de escala

Q5.1. Para $n \in \{10, 20, 50, 100, 200, 500\}$, meça o tempo de execução de `resolver_gauss` com matrizes aleatórias. Ajuste uma lei de potência $T(n) = c \cdot n^\alpha$ usando `numpy.polyfit` no espaço log-log. O expoente α encontrado é próximo de 3?

Q5.2. Compare o tempo medido com a previsão teórica $T_{\text{teo}}(n) \approx \frac{2n^3}{3}/R$, onde R é o número de operações de ponto flutuante por segundo do seu computador (estime R medindo o tempo de n^3 multiplicações em um laço Python). Qual é a eficiência relativa $T_{\text{med}}/T_{\text{teo}}$? Por que é menor que 1?

5.2. Comparação de métodos

Q5.3. Para $n = 300$ e uma matriz SPD aleatória, compare os tempos de: Gauss com pivoteamento, LU (Doolittle), Cholesky e `numpy.linalg.solve`. Organize numa tabela com tempo, flops teóricos e razão em relação a Cholesky. O que explica a diferença entre implementações puras em Python e NumPy?

6. Condicionamento e Sensibilidade à Perturbação

```

1 import numpy as np
2 from scipy.linalg import hilbert    # matriz de Hilbert
3
4 def experimento_hilbert(n):
5     """Resolve  $H_n x = b$  com  $b = H_n @ \text{ones}$  e mede o erro."""
6     H = hilbert(n)
7     x_exato = np.ones(n)
8     b = H @ x_exato
9     x_calc = np.linalg.solve(H, b)
10    erro_rel = np.linalg.norm(x_calc - x_exato) / np.linalg.norm(x_exato)
11    kappa = np.linalg.cond(H)
12    return kappa, erro_rel
13
14 def perturbar_b(A, b, nivel=1e-6, n_amstras=100):
15     """Aplica perturbações aleatórias a b e mede a amplificação do erro."""
16     x_exato = np.linalg.solve(A, b)
17     amplificacoes = []
18     for _ in range(n_amstras):
19         db = nivel * np.random.randn(len(b))
20         x_pert = np.linalg.solve(A, b + db)
21         amp = (np.linalg.norm(x_pert - x_exato) / np.linalg.norm(x_exato)) / \
22             (np.linalg.norm(db) / np.linalg.norm(b))
23         amplificacoes.append(amp)
24     return np.array(amplificacoes)

```

Listing 5: condicao.py — experimentos de condicionamento

6.1. Matriz de Hilbert

Q6.1. Para $n = 4, 6, 8, 10, 12$, execute `experimento_hilbert(n)`. Preencha e analise a tabela:

n	$\kappa_2(H_n)$	Erro relativo	Dígitos corretos ($\approx 16 - \log_{10} \kappa$)
4			
6			
8			
10			
12			

A partir de qual n o resultado se torna completamente não confiável?

6.2. Amplificação de erros

Q6.2. Use `perturbar_b` com $A = H_6$ e compare com $A = I_6$. Para cada matriz, plote o histograma das amplificações. Qual é a amplificação máxima observada? Ela está dentro do limite $\kappa(A) \cdot \|\delta b\|/\|b\|$?

Q6.3. Gere duas matrizes 5×5 : uma bem-condicionada ($\kappa \approx 1$, e.g., matriz diagonal com entradas próximas) e uma mal-condicionada ($\kappa > 10^6$, e.g., Hilbert H_5). Para cada uma, introduza um erro de *apenas* 10^{-6} nos dados de A e meça o impacto na solução. Discuta as implicações práticas para aplicações de engenharia.

7. Projeto Integrador: PageRank Numérico

Esta seção integra os métodos estudados numa aplicação real de relevância histórica.

Contexto. O algoritmo PageRank, usado pelo Google, atribui importância às páginas web resolvendo o sistema $(I - \alpha P^T)\pi = \frac{1-\alpha}{n}\mathbf{e}$, onde P é a matriz de transição, $\alpha \approx 0,85$ é o fator de amortecimento e π é o vetor de ranks. Note que $I - \alpha P^T$ é uma matriz densa e *geralmente* bem-condicionada.

Q7.1. Para a mini-rede de 4 páginas da aula (matriz de adjacência G fornecida), implemente o cálculo do PageRank:

- Construa a matriz de transição P normalizando as colunas de G ;
- Monte o sistema $(I - 0,85 P^T)\pi = 0,15/4 \cdot \mathbf{e}$;
- Resolva com `resolver_gauss` e com `resolver_lu`;
- Verifique que $\|\pi\|_1 = 1$ (distribuição de probabilidade);
- Ordene as páginas por rank decrescente.

Q7.2. Amplie a rede para $n = 20$ páginas com grafo aleatório (probabilidade de aresta $p = 0,3$, `np.random.seed(0)`). Compare os ranks obtidos por Gauss e por LU com `scipy.linalg.lu_solve`. Meça o tempo e o resíduo de cada abordagem.

Q7.3. O que acontece com $(I - \alpha P^T)$ quando $\alpha \rightarrow 1$? Calcule $\kappa_2(I - \alpha P^T)$ para $\alpha \in \{0,5, 0,7, 0,85, 0,95, 0,99\}$ e plote. Por que $\alpha = 0,85$ é considerado um bom compromisso?

8. Desafio (Opcional — Pontuação Extra)

Desafio Computacional

As questões desta seção são opcionais e destinadas a alunos que desejam aprofundar a investigação.

Q8.1. Análise de estabilidade em cascata. Construa uma sequência de matrizes A_ε que dependem de um parâmetro $\varepsilon > 0$ tal que A_0 é singular. Investigue como o resíduo e o erro da solução crescem conforme $\varepsilon \rightarrow 0$. Relacione com $\kappa(A_\varepsilon)$.

Q8.2. Bloco tridiagonal e EDPs 2D. A discretização por diferenças finitas do problema de Poisson em grade $m \times m$ produz uma matriz bloco-tridiagonal de tamanho $n = m^2$. Para $m = 10$, construa esta matriz, resolva o sistema com Gauss e com `scipy.sparse.linalg.spsolve` (formato esparsa). Compare tempos e memória.

Q8.3. Implementação vetorizada. Reescreva `fatoracao_lu` usando apenas operações vetorizadas do NumPy (sem laços Python explícitos). Meça o ganho de desempenho para $n = 100$.

Entrega e Critérios de Avaliação

O que entregar

Formato de entrega (via Moodle, antes do prazo):

1. **Relatório PDF** com todas as respostas, tabelas preenchidas, gráficos gerados e análises escritas. Gráficos devem ter título, eixos rotulados e legenda.
2. **Arquivos Python** organizados: `gauss.py`, `lu.py`, `cholesky.py`, `thomas.py`, `condicao.py`, `pagerank.py` e `main.py` (script que reproduz todos os resultados do relatório).
3. Cada arquivo deve conter uma **docstring** descrevendo o que faz.

Seção	Questões	Valor
1. Eliminação de Gauss	Q1.1–Q1.5	2,0 pts
2. Fatoração LU	Q2.1–Q2.4	2,0 pts
3. Fatoração de Cholesky	Q3.1–Q3.3	2,0 pts
4. Algoritmo de Thomas	Q4.1–Q4.3	1,5 pts
5. Custo computacional	Q5.1–Q5.3	1,0 pts
6. Condicionamento	Q6.1–Q6.3	1,5 pts
7. Projeto PageRank	Q7.1–Q7.3	1,5 pts
Total obrigatório		11,5 pts
8. Desafio (extra)	Q8.1–Q8.3	até 1,5 pts

Critérios de qualidade:

- Código limpo, comentado e executável sem erros;
- Análises escritas com rigor: não basta dizer “o erro aumentou”, é preciso quantificar e justificar;
- Gráficos com qualidade (títulos, legendas, escala adequada);
- Conexão entre resultados empíricos e teoria apresentada em aula.

A. Esqueleto do Script Principal

O código abaixo serve como ponto de partida para `main.py`. Preencha as seções marcadas com `# TODO`.

```

1  """
2  main.py
3  Plano de Investiga o      Sistemas Lineares
4  Disciplina: C lculo Num rico
5  """
6  import numpy as np
7  import time
8  import matplotlib.pyplot as plt
9  from gauss      import resolver_gauss
10 from lu         import fatoracao_lu, subst_prog, subst_retro
11 from cholesky   import resolver_cholesky
12 from thomas     import thomas, montar_tridiagonal
13 from condicao    import experimento_hilbert, perturbar_b
14
15 #              Se o 1: Verifica o b sica
16
17 A = np.array([[3, 2,  4],
18              [1, 1,  2],
19              [4, 3, -2]], dtype=float)
20 b = np.array([1, 2, 3], dtype=float)
21
22 x = resolver_gauss(A, b)
23 print(f"Se o 1      Solu o: {x}")
24 print(f"Se o 1      Res duo: {np.linalg.norm(A @ x - b):.2e}"
25       )
26
27 #              TODO: Se o 2      Efeito do pivoteamento
28
29 # (implemente aqui o experimento com float32)
30
31 #              TODO: Se o 3      Fatora o LU, m ltiplos b
32
33
34 #              TODO: Se o 4      Thomas vs. Gauss
35
36
37 #              TODO: Se o 5      Custo computacional
38
39
40 #              TODO: Se o 6      Condicionamento

```

```
35 for n in [4, 6, 8, 10, 12]:
36     kappa, erro = experimento_hilbert(n)
37     print(f"n={n:2d}   kappa={kappa:.2e}   erro={erro:.2e}")
38
39 #           TODO: Se o 7 PageRank
40
41 plt.tight_layout()
42 plt.savefig("resultados.pdf", dpi=150)
43 plt.show()
```

Listing 6: main.py — esqueleto para o relatório

B. Referências Bibliográficas

- [1] Trefethen, L. N.; Bau, D. *Numerical Linear Algebra*. SIAM, 1997.
- [2] Golub, G. H.; Van Loan, C. F. *Matrix Computations*. 4.^a ed. JHU Press, 2013.
- [3] Burden, R. L.; Faires, J. D. *Numerical Analysis*. 10.^a ed. Cengage, 2015. Cap. 6.
- [4] NumPy — <https://numpy.org/doc/stable/reference/routines.linalg.html>
- [5] SciPy — <https://docs.scipy.org/doc/scipy/reference/linalg.html>